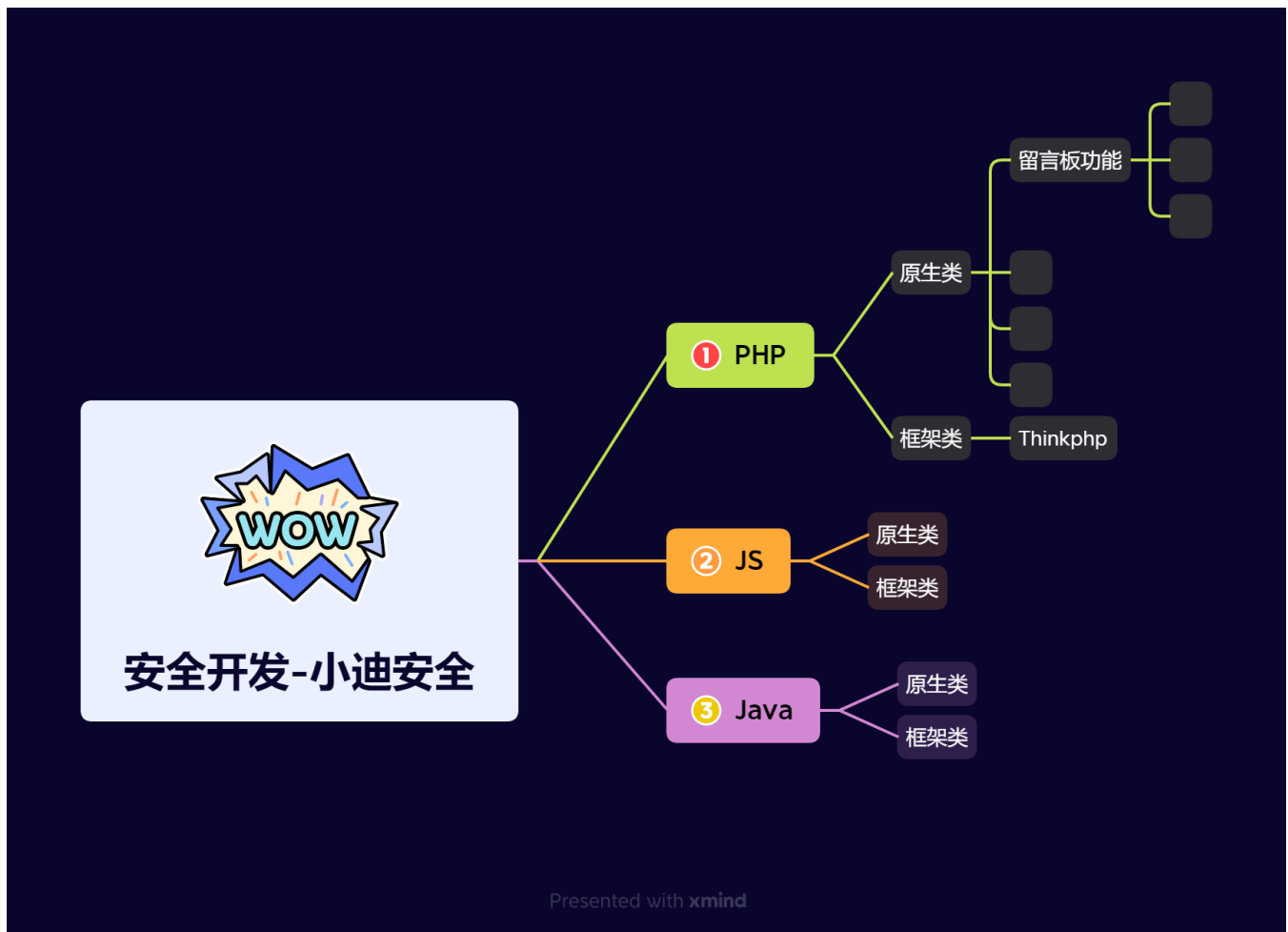


Day33 安全开发-JavaEE应用&SQL预编译&Filter过滤器&Listener监听器&访问控制



1.知识点

- 1、PHP留言板前后端功能实现
- 2、数据库创建&架构&增删改查
- 3、内置超全局变量&HTML&JS混编
- 4、第三方应用插件&传参&对象调用

-
- 1、PHP后台身份验证模块实现
 - 2、Cookie&Session技术&差异
 - 3、Token数据包唯一性应用场景
 - 项目1：用cookie做后台身份验证
 - 项目2：用session做后台身份验证
 - 项目3：用token做用户登录判断
-

- 1、PHP文件管理-显示&上传功能实现
 - 2、文件上传-\$_FILES&过滤机制实现
 - 3、文件显示-目录遍历&过滤机制实现
-

- 1、PHP文件管理-下载&删除功能实现
 - 2、PHP文件管理-编辑&包含功能实现
-

- 1、PHP新闻显示-数据库操作读取显示
 - 2、PHP模版引用-自写模版&Smarty渲染
 - 3、PHP模版安全-RCE代码执行&三方漏洞
-

- 1、TP框架-开发-路由访问&数据库&文件上传&MVC模型
 - 2、TP框架-安全-不合规写法&内置过滤绕过&版本安全漏洞
-

- 1、JS应用-原生态开发&第三库开发
 - 2、JS功能-文件上传&登录验证&商品购买
-

- 1、JS技术-DOM树操作及安全隐患
 - 2、JS技术-加密编码及数据安全调试
-

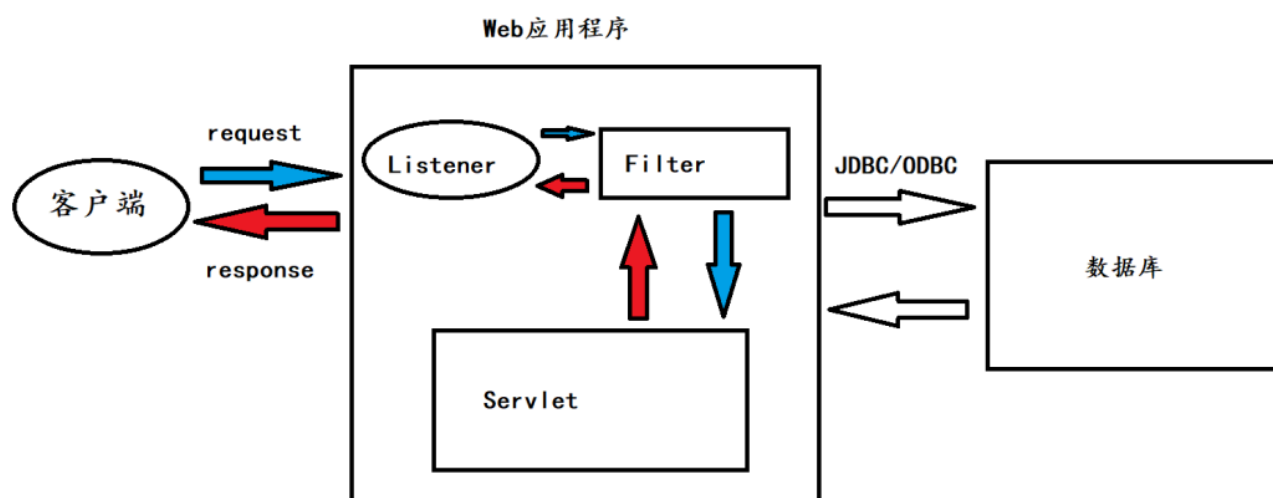
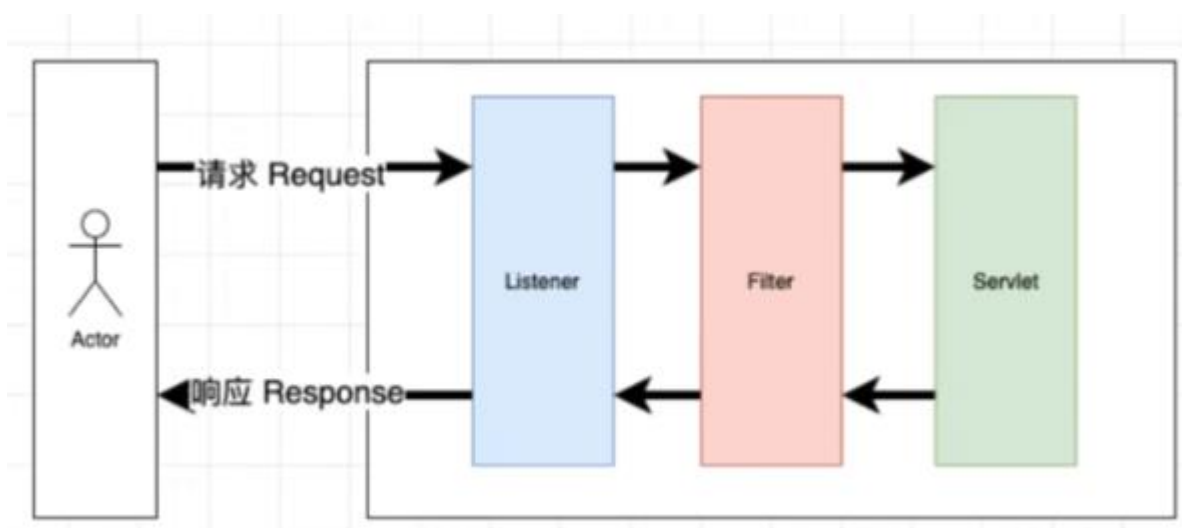
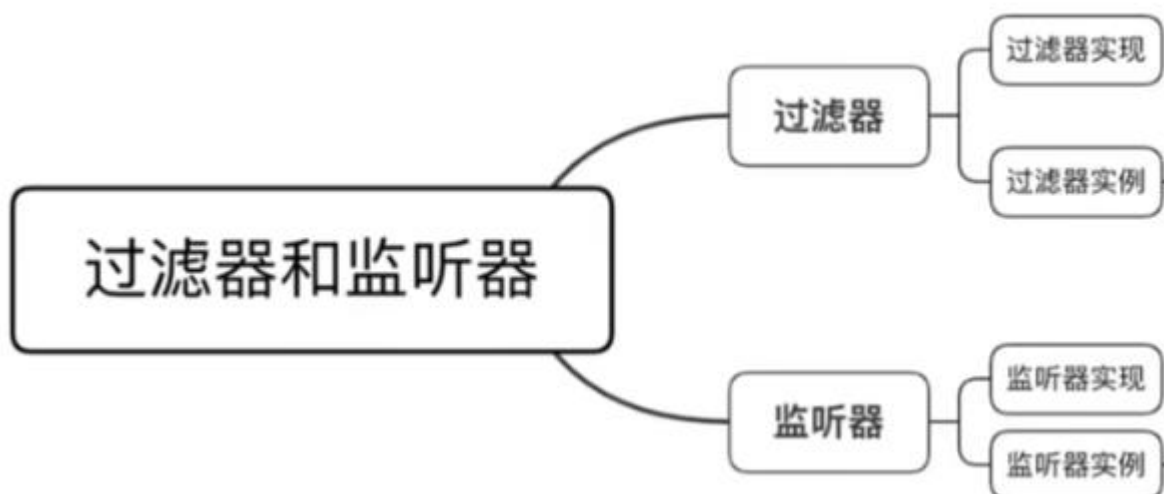
- 1、NodeJS-开发环境&功能实现
 - 2、NodeJS-安全漏洞&案例分析
 - 3、NodeJS-开发指南&特有漏洞
-

- 1、三方库-JQuery-使用&安全
 - 2、打包器-Webpack-使用&安全
-

- 1、JavaEE-HTTP-Servlet技术
 - 2、JavaEE-数据库-JDBC&Mybatis
-

- 1、JavaEE-JDBC-SQL预编译
 - 2、JavaEE-HTTP-Filter过滤器
 - 3、JavaEE-对象域-Listen监听器
-

2.演示案例



2.1 JavaEE-预编译-SQL

- 1 预编译SQL语句并执行,预防SQL注入问题
- 2 `String safesql="select * from news where id=?";`
- 3 `PreparedStatement preparedStatement=connection.prepareStatement();`
- 4 `preparedStatement.setString(1,s);`
- 5 `ResultSet resultSet=preparedStatement.executeQuery();`

(1) 未使用预编译的sql语句:

```
10 Scanner scanner=new Scanner(System.in);
11 System.out.print("please input id:");
12 String s=scanner.nextLine();
13 System.out.println(s);
14
15
16
17 Class.forName(className: "com.mysql.jdbc.Driver");
18 String url = "jdbc:mysql://localhost:3306/demo01";
19 Connection connection = DriverManager.getConnection(url, user: "root", password: "123456");
20 System.out.println(connection);
21
22 String sql="select * from news where id="+s;
23 System.out.println(sql);
24 Statement statement= connection.createStatement();
25 ResultSet resultSet = statement.executeQuery(sql);
26
27 // 预编译写法
28 String safesql="select * from news where id=?";
29 PreparedStatement preparedStatement=connection.prepareStatement(safesql);
30 preparedStatement.setString(1,s);
31 ResultSet resultSet=preparedStatement.executeQuery();
32 System.out.println(safesql);
33
34 while (resultSet.next()){
35
36 }
```

直接将用户输入拼接查询

```
18 Class.forName(className: "com.mysql.jdbc.Driver");
19 String url = "jdbc:mysql://localhost:3306/demo01";
20 Connection connection = DriverManager.getConnection(url, user: "root", password: "123456");
21 System.out.println(connection);
22
23 // 不安全的写法
24 String sql="select * from news where id="+s;
25 System.out.println(sql);
26 Statement statement= connection.createStatement();
27 ResultSet resultSet = statement.executeQuery(sql);
28
29 // 预编译写法
```

拼接语句以及查询结果


```
import java.sql.*;
import java.util.Scanner;

public class NewsServlet {
    public static void main(String[] args) throws ClassNotFoundException, SQLException {
        //接受用户输入的 变量字符串s接受
        Scanner scanner=new Scanner(System.in);
        System.out.print("please input id:");
        String s=scanner.nextLine();

        // 1 union select 1,2,3,version(),user(),database()
        String sql = "select * from news where id=" + s;
        Connection connection = DriverManager.getConnection("jdbc:mysql://localhost:3306/test", "root", "123456");
        Statement statement = connection.createStatement();
        ResultSet resultSet = statement.executeQuery(sql);

        while(resultSet.next()){
            System.out.println(resultSet.getString(1) + "|" + resultSet.getString(2) + "|" + resultSet.getString(3) + "|" + resultSet.getString(4) + "|" + resultSet.getString(5));
        }
    }
}
```

运行结果:

```
please input id: 1 union select 1,2,3,version(),user(),database()
1 union select 1,2,3,version(),user(),database()
Sat Apr 29 20:18:37 CST 2023 WARN: Establishing SSL connection without server's identity verification is not recommended. According
com.mysql.jdbc.JDBC4Connection@22d8cfe0
select * from news where id=1 union select 1,2,3,version(),user(),database()
1|小迪安全博客|小迪培训开课了|V2021-3冲!|好瓜啊|img/xiaodi.png
1|2|3|5.7.26|root@localhost|demo01
```

进行sql注入，查询到的结果

(2) 使用预编译的sql语句:

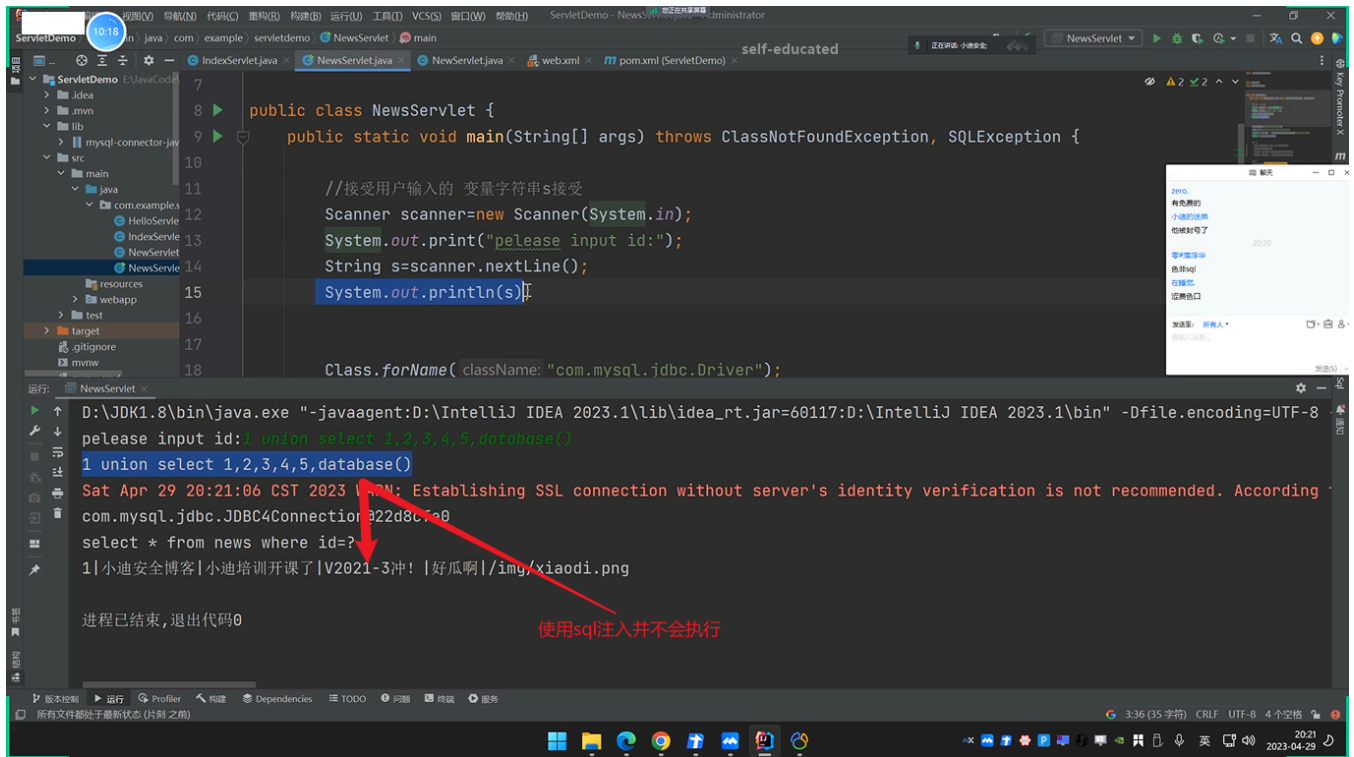
```
String sql="select * from news where id="+s;
System.out.println(sql);
Statement statement= connection.createStatement();
ResultSet resultSet = statement.executeQuery(sql);

// 预编译写法
String safesql="select * from news where id=?";
PreparedStatement preparedStatement=connection.prepareStatement(safesql);
preparedStatement.setString( parameterIndex: 1,s);
ResultSet resultSet=preparedStatement.executeQuery();
System.out.println(safesql);
```

运行结果:

```
please input id: 1 union select 1,2,3,version(),user(),database()
1 union select 1,2,3,version(),user(),database()
Sat Apr 29 20:18:37 CST 2023 WARN: Establishing SSL connection without server's identity verification is not recommended. According
com.mysql.jdbc.JDBC4Connection@22d8cfe0
select * from news where id=1 union select 1,2,3,version(),user(),database()
1|小迪安全博客|小迪培训开课了|V2021-3冲!|好瓜啊|img/xiaodi.png
1|2|3|5.7.26|root@localhost|demo01
```

预编译写法



2.2 JavaEE-过滤器-Filter

1 **Filter**被称为过滤器，过滤器实际上就是对web资源进行拦截，做一些处理后再交给下一个过滤器或Servlet处理，通常都是用来拦截request进行处理的，也可以对返回的response进行拦截处理。开发人员利用filter技术，可以实现对所有web资源的管理，例如实现权限访问控制、过滤敏感词汇、压缩响应信息等一些高级功能。

2 1、创建过滤器

3 2、过滤器内置方法

4 init doFilter destroy

5 3、过滤器触发流程

6 @WebFilter("/xss")

7 <filter>

8 <filter-name>xssFilter</filter-name>

9 <filter-class>com.example.filter.xssFilter</filter-class>

10 </filter>

11 <filter-mapping>

12 <filter-name>xssFilter</filter-name>

13 <url-pattern>/xss</url-pattern>

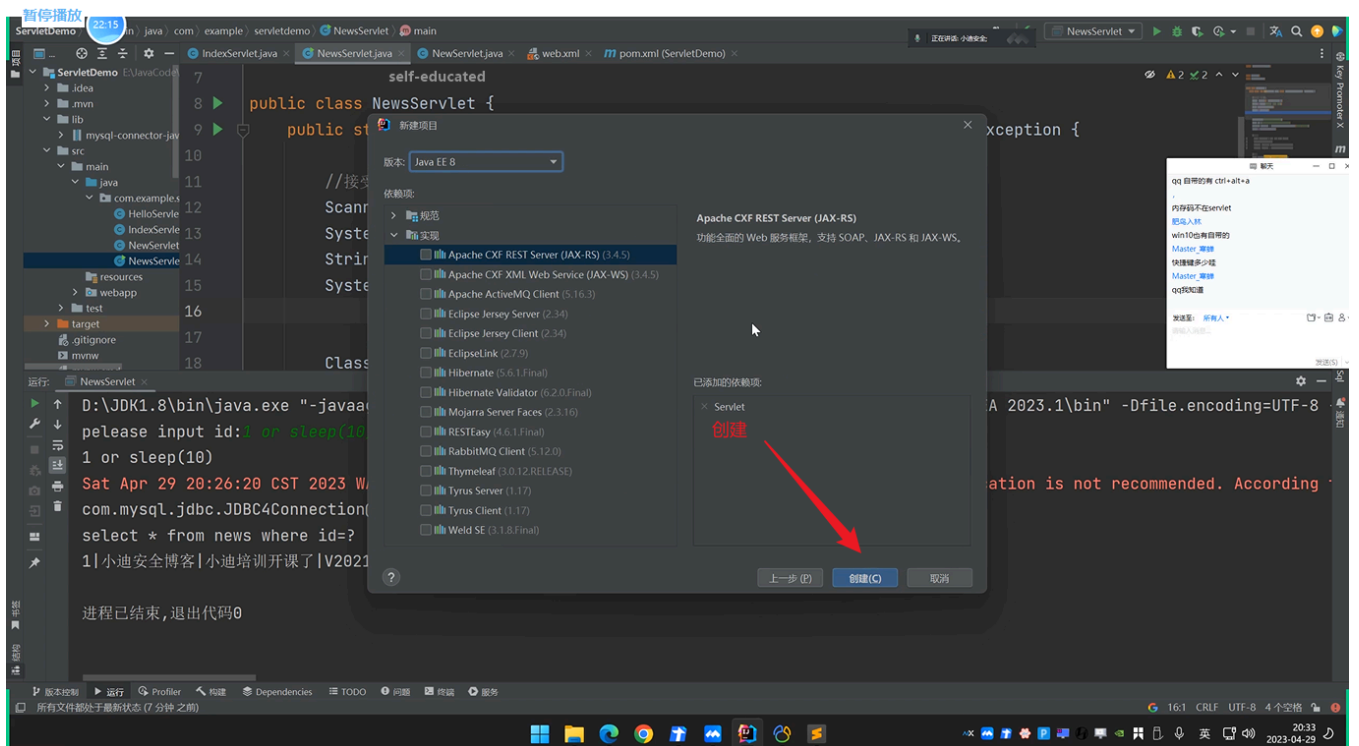
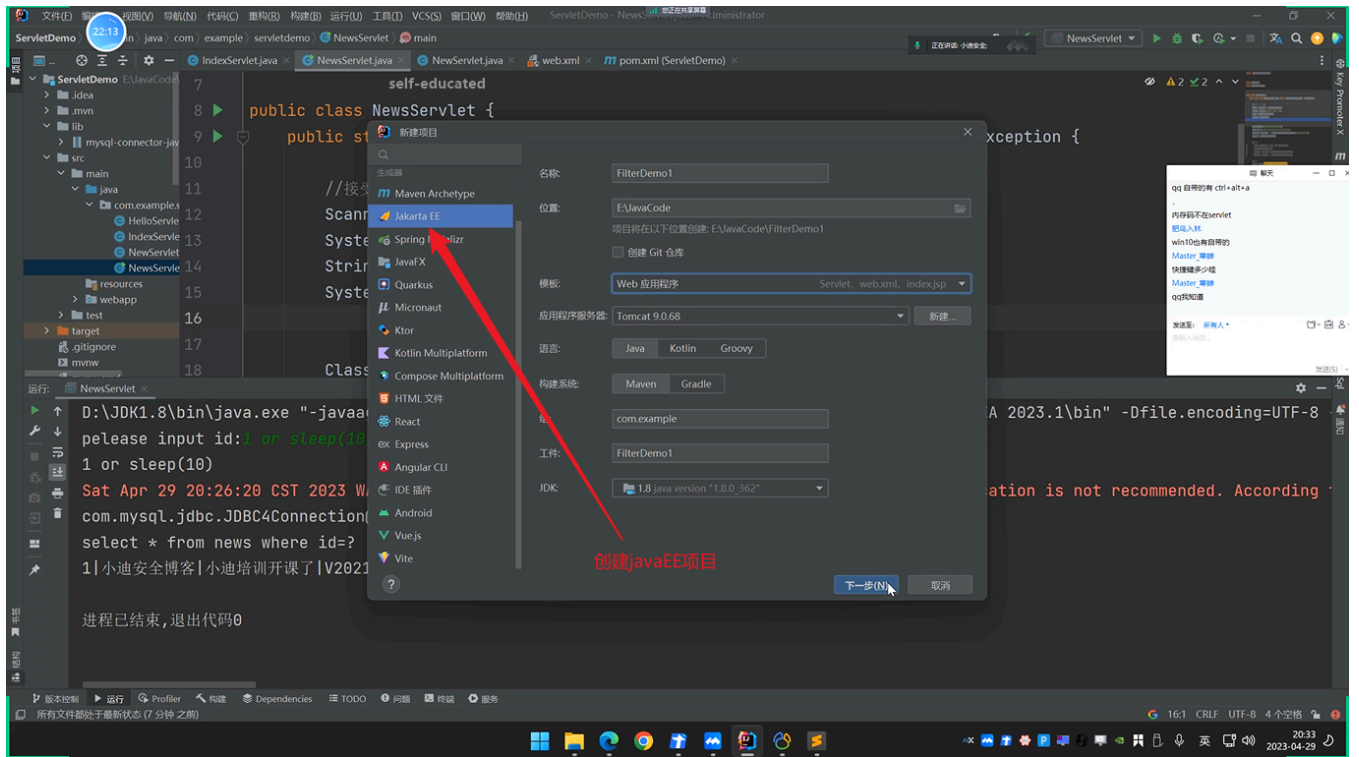
14 </filter-mapping>

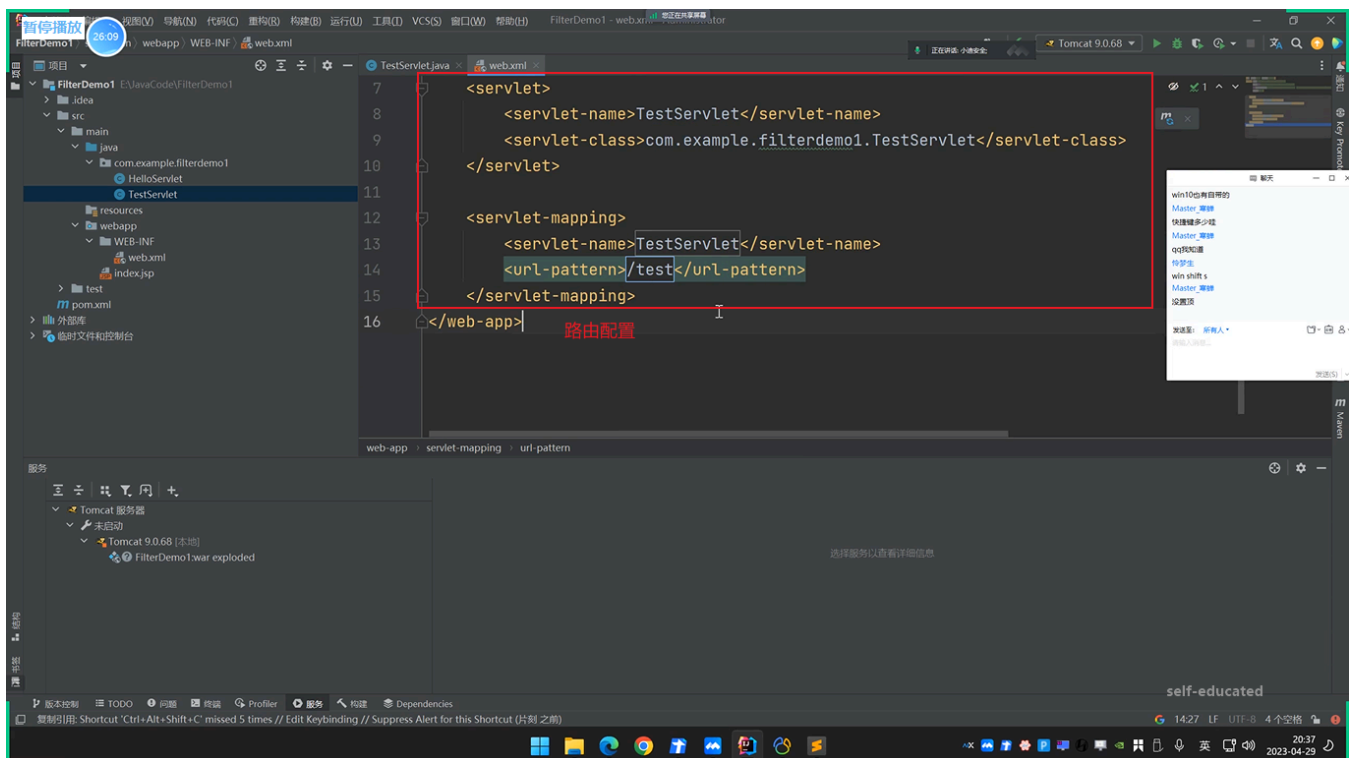
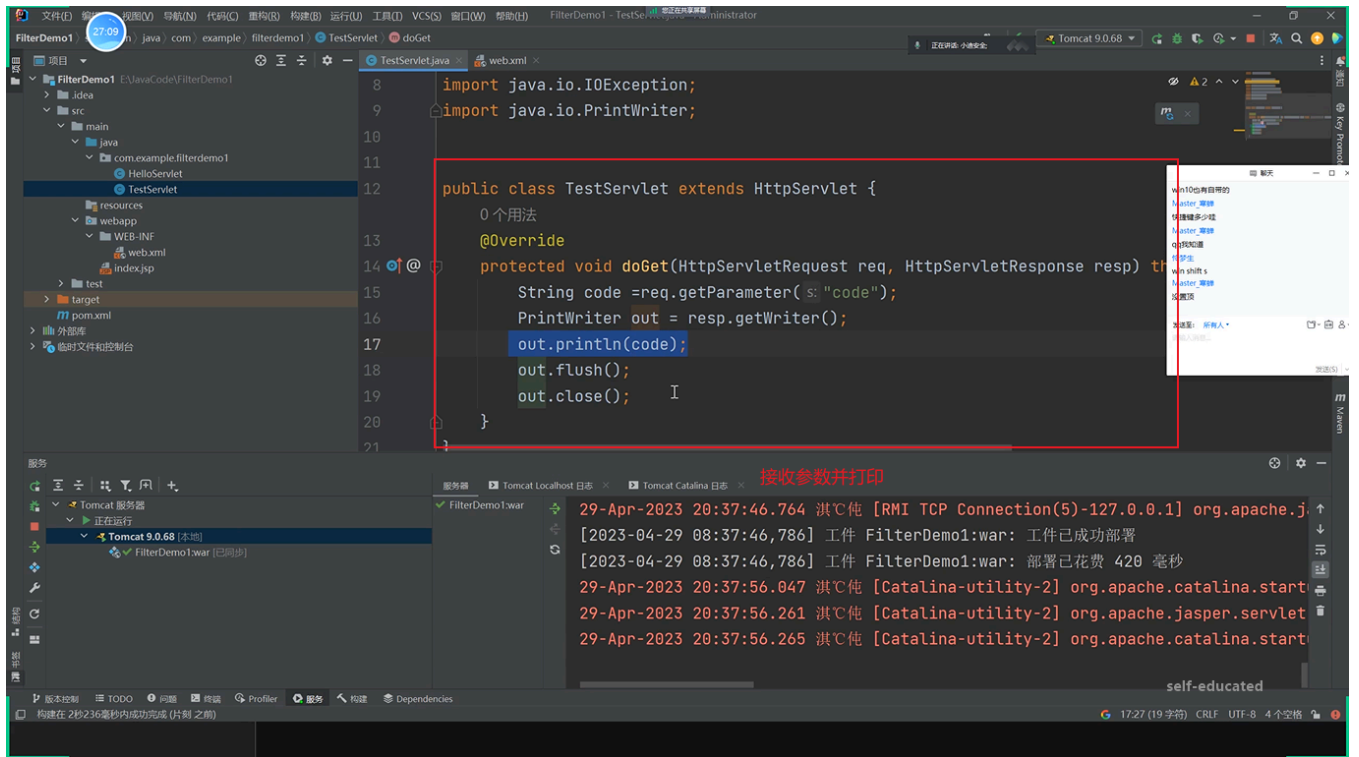
15 4、过滤器安全场景

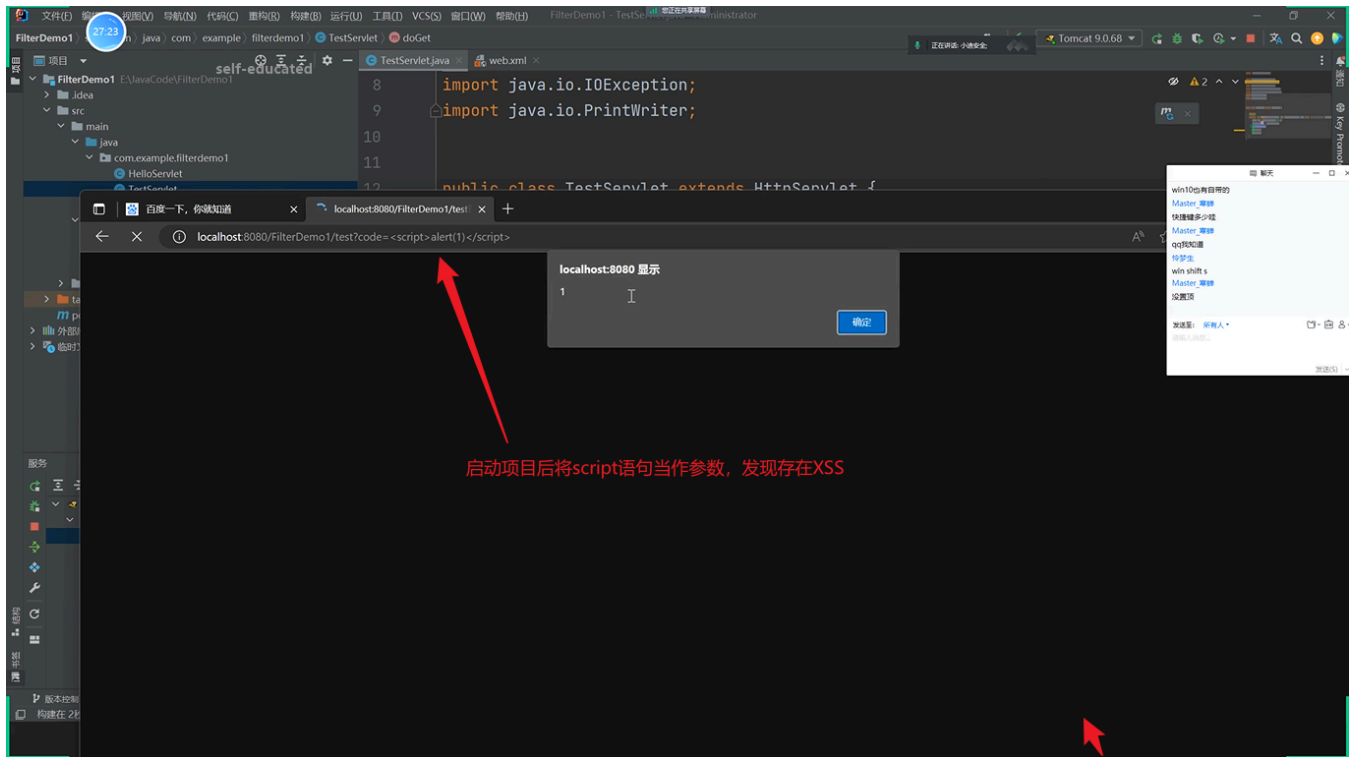
16 Payload检测，权限访问控制，红队内存马植入，蓝队清理内存马等

17 内存马参考: <https://mp.weixin.qq.com/s/hev4G1FivLtqKjt0VhHKmw>

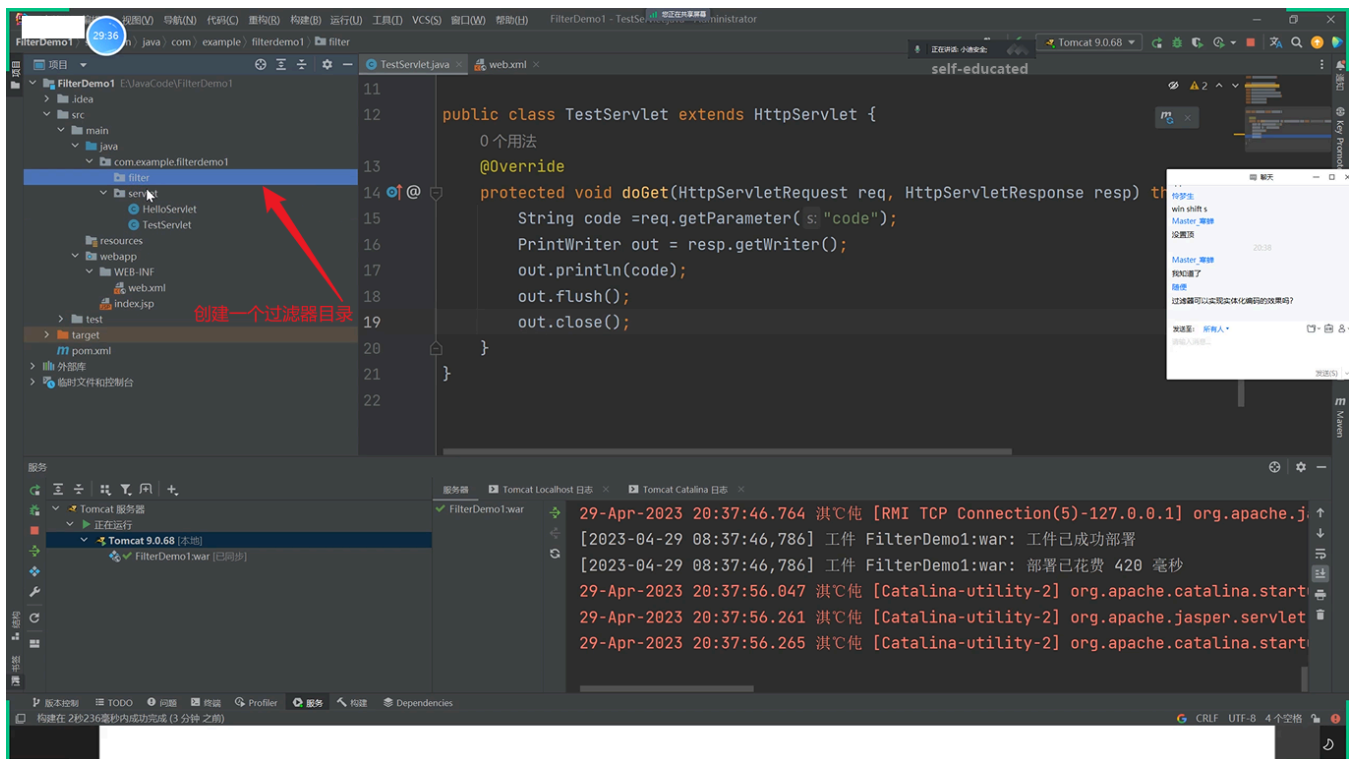
(1) 创建过滤器:

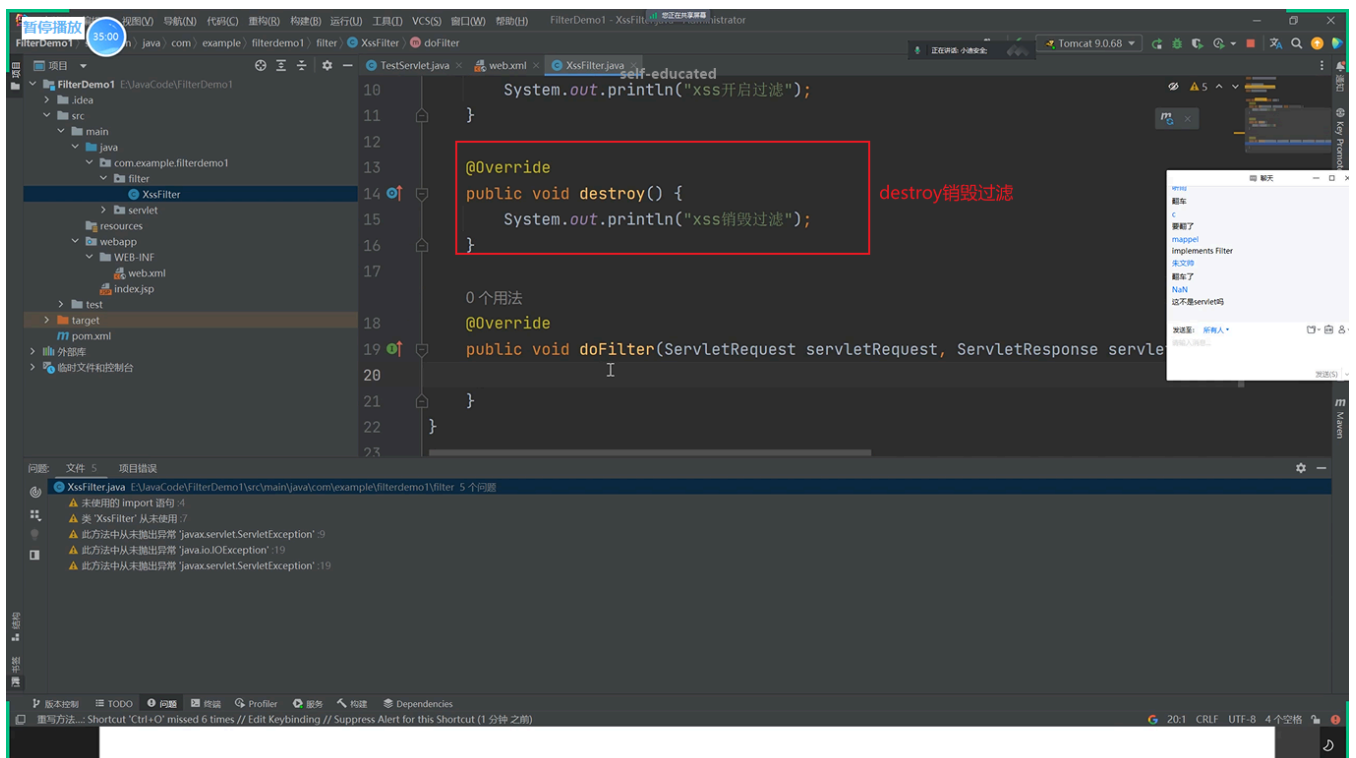
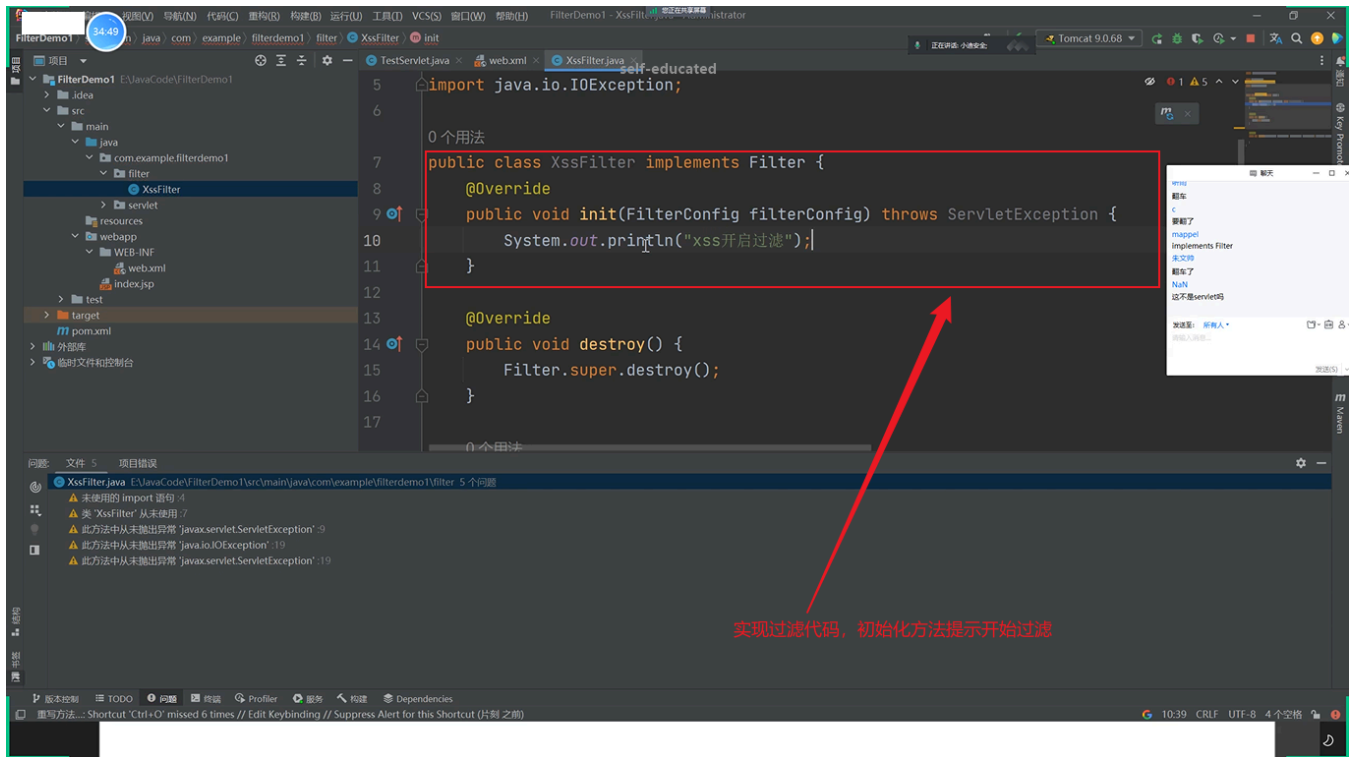


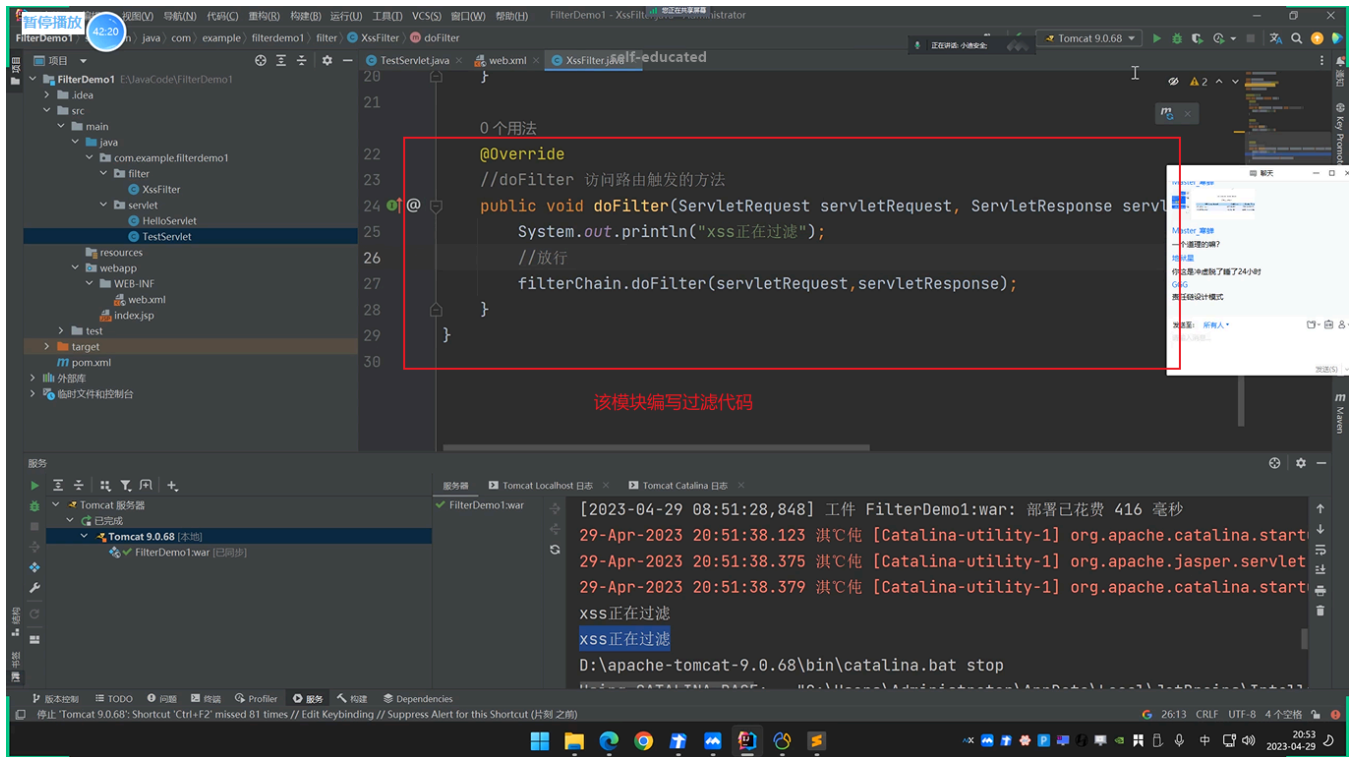




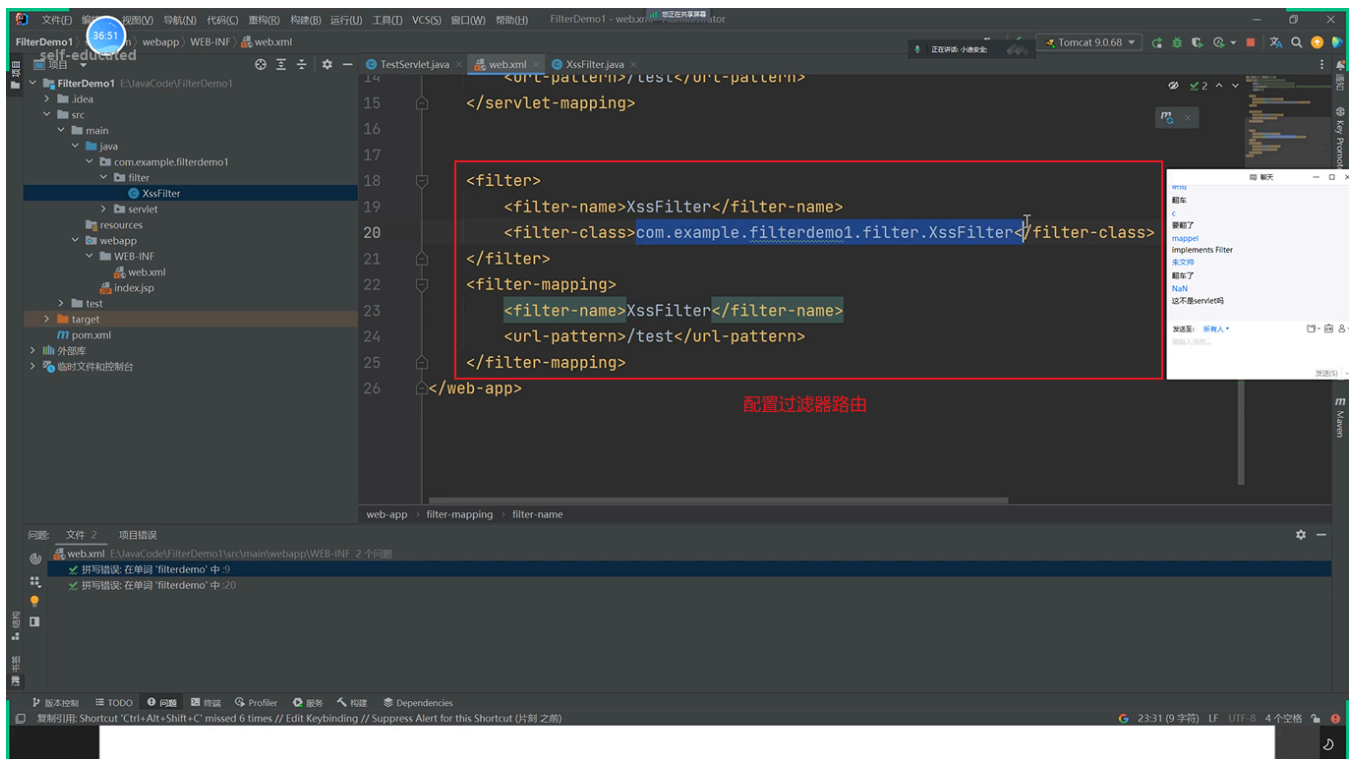
(2) 过滤器内置方法:

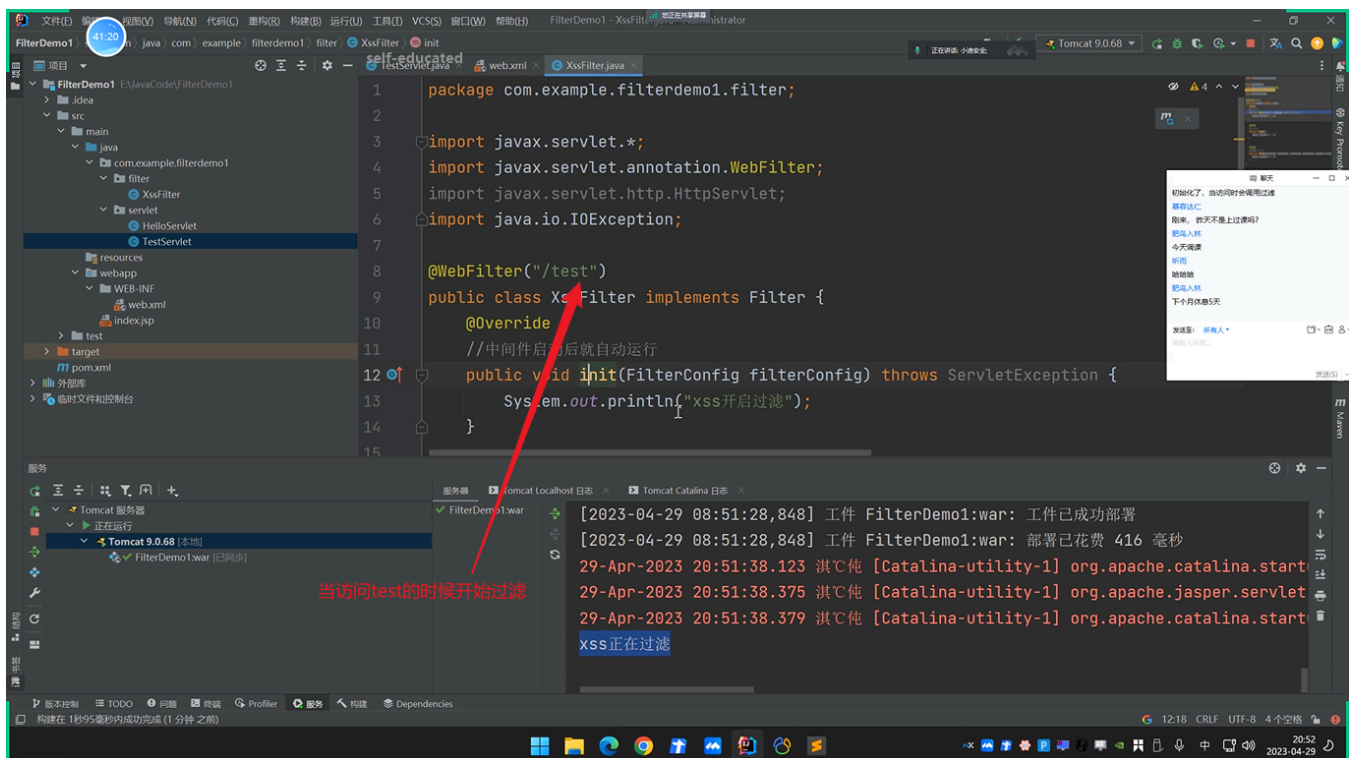
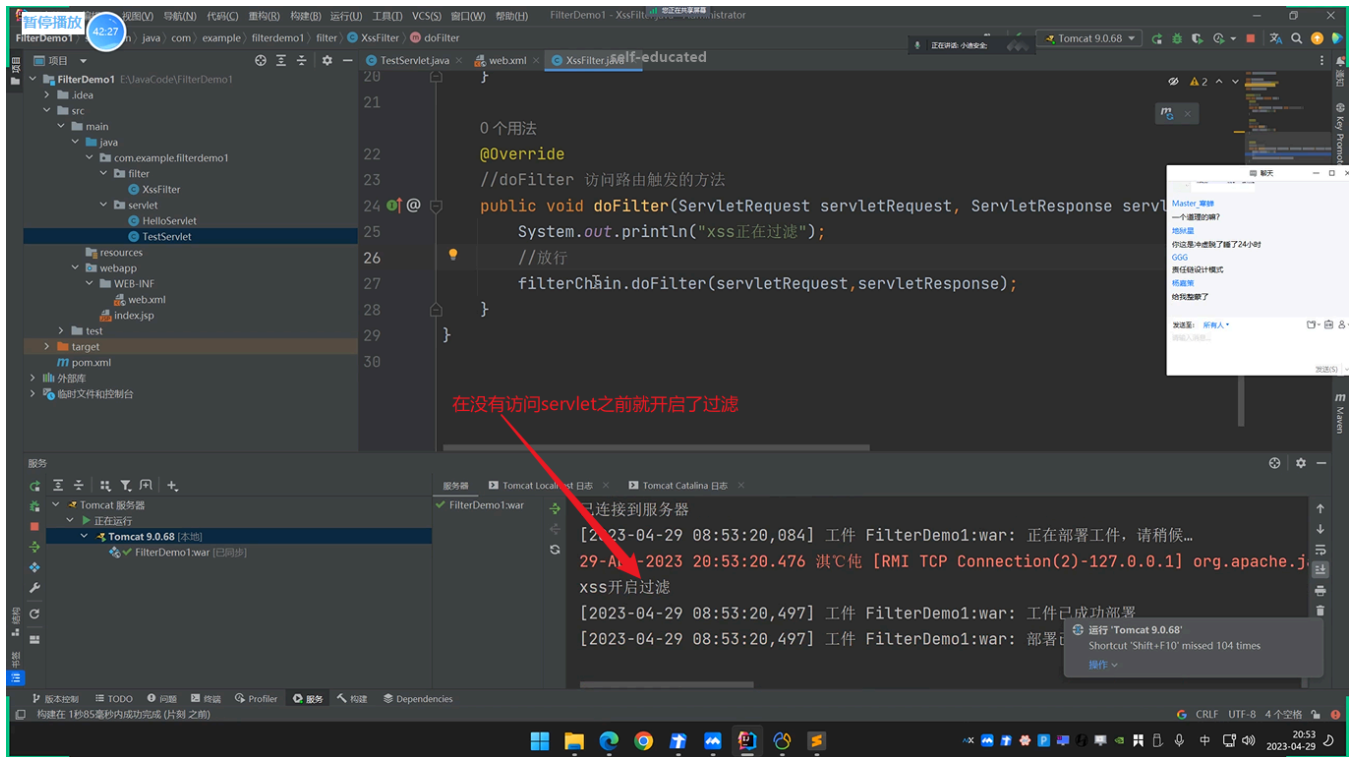




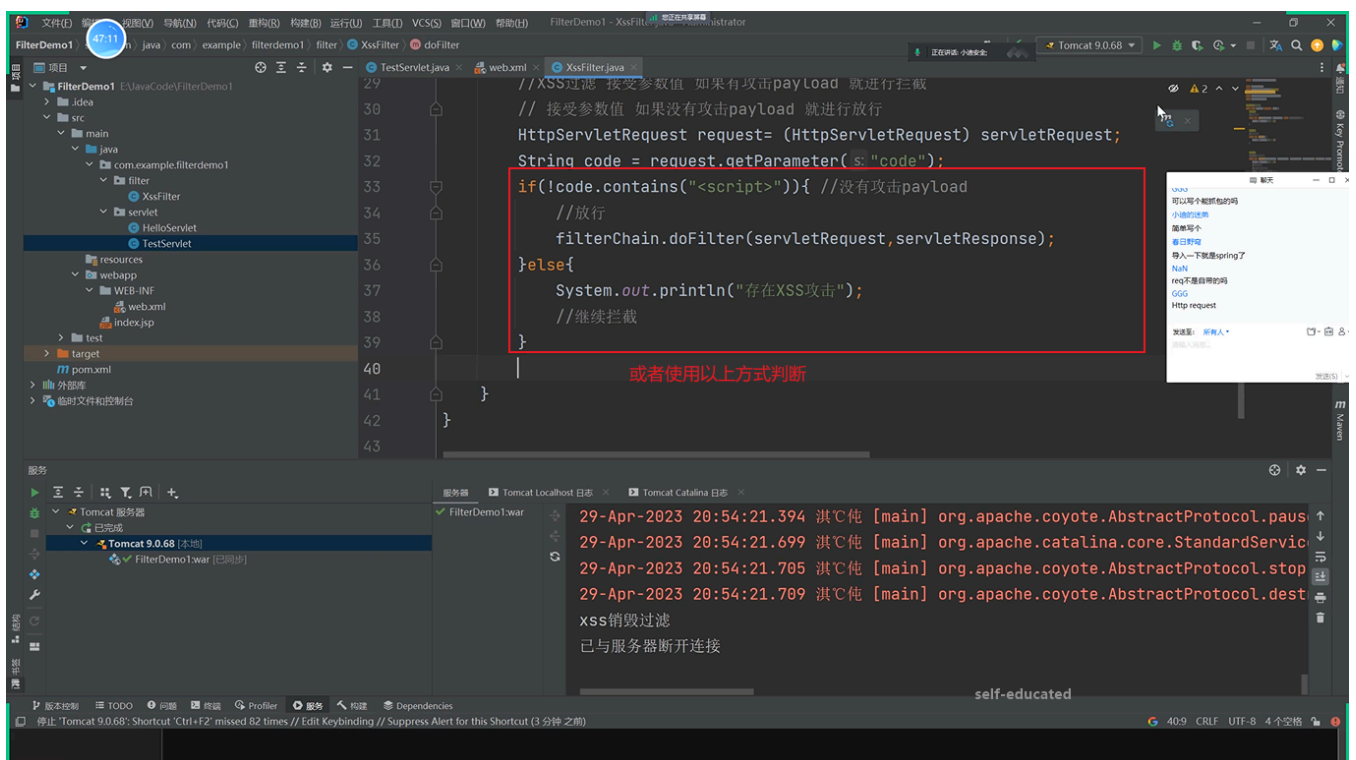
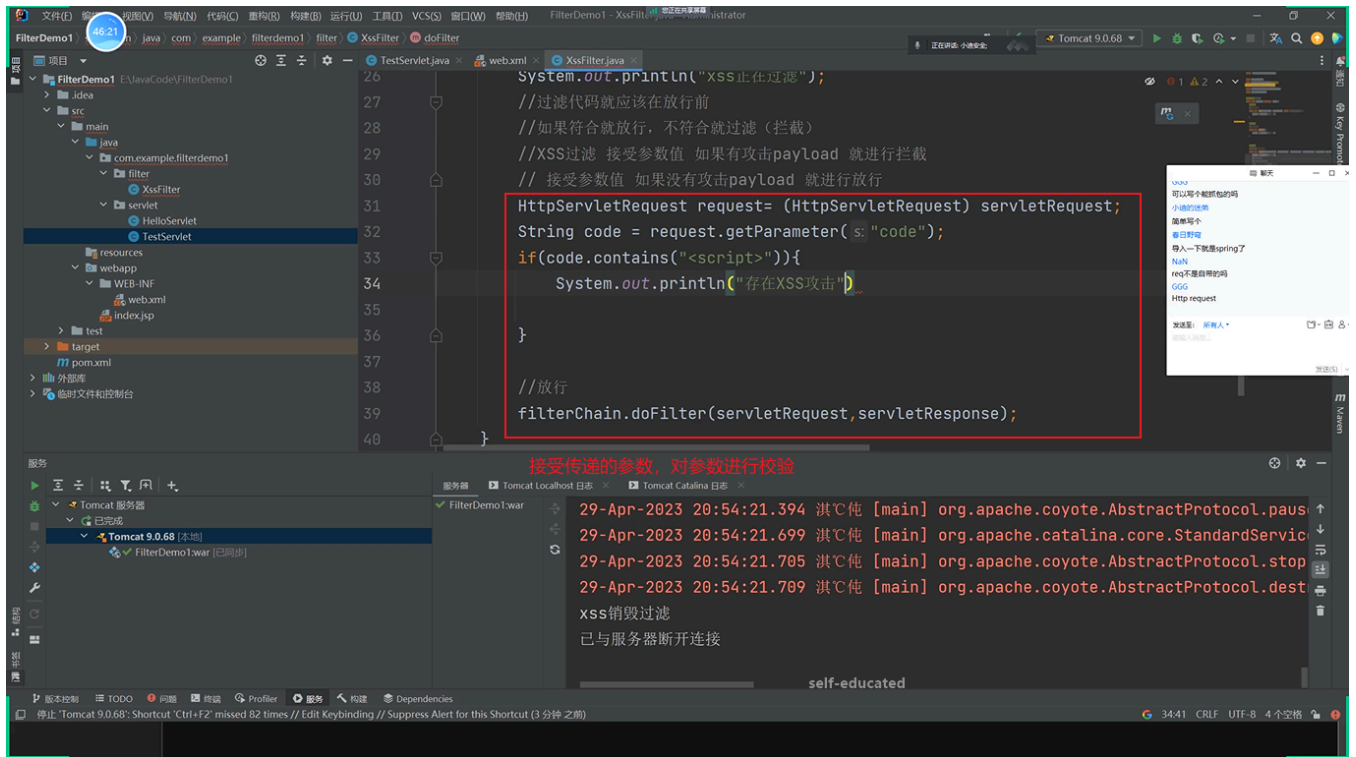


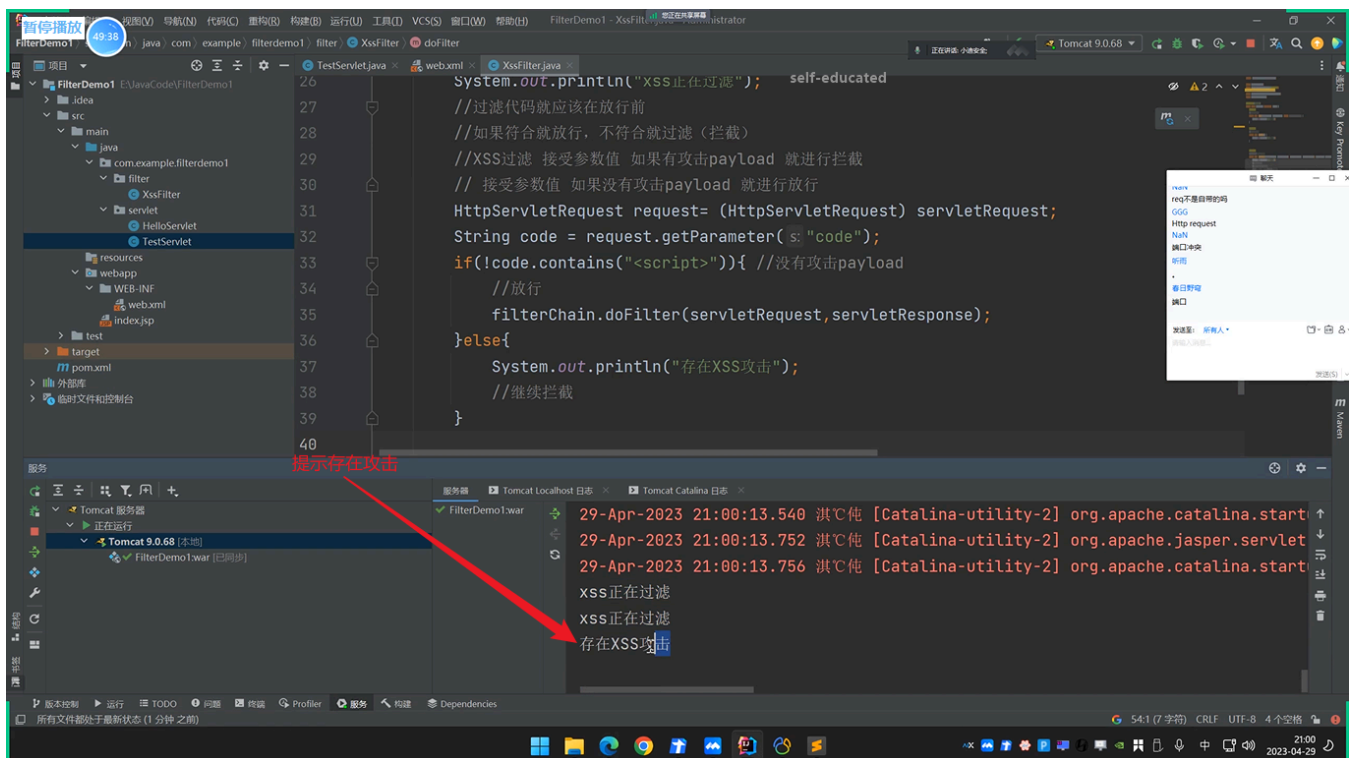
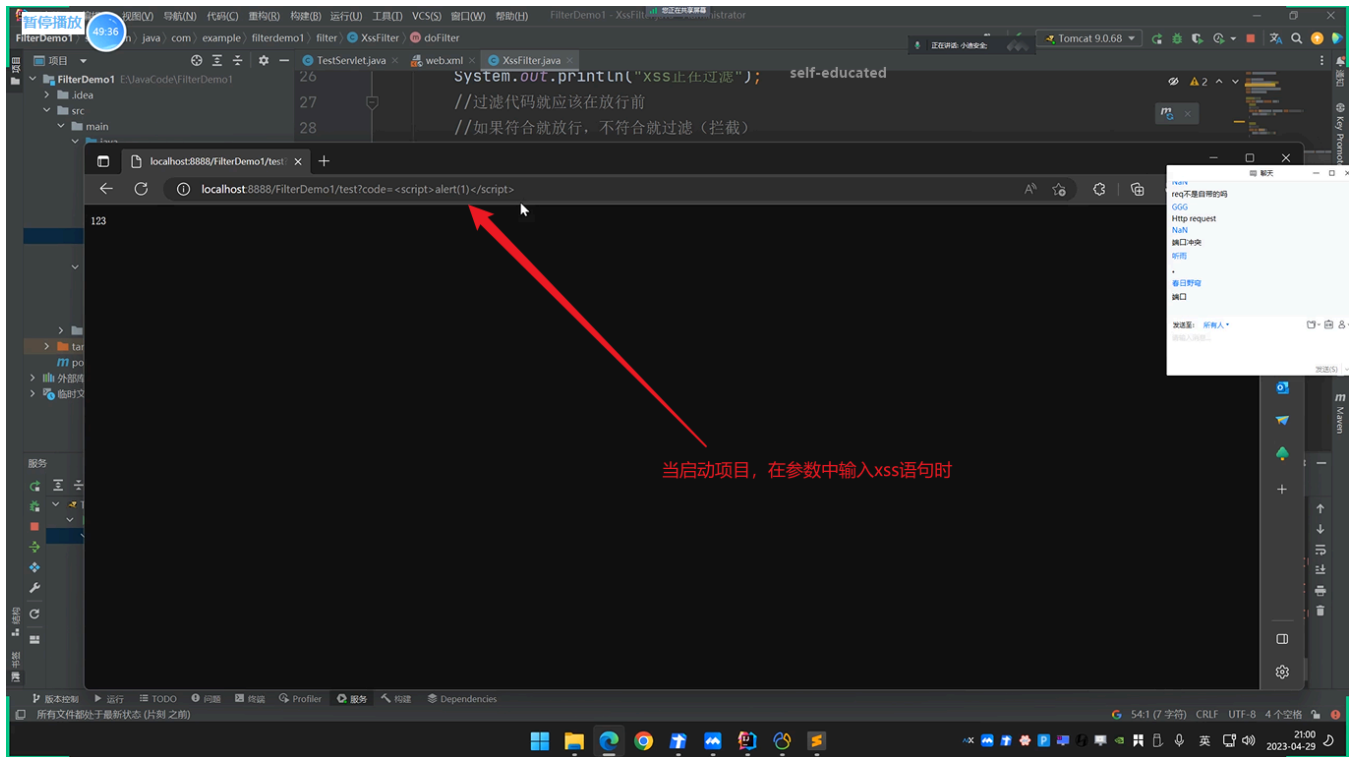
(3) 过滤器触发流程:





(4) 编写过滤规则:

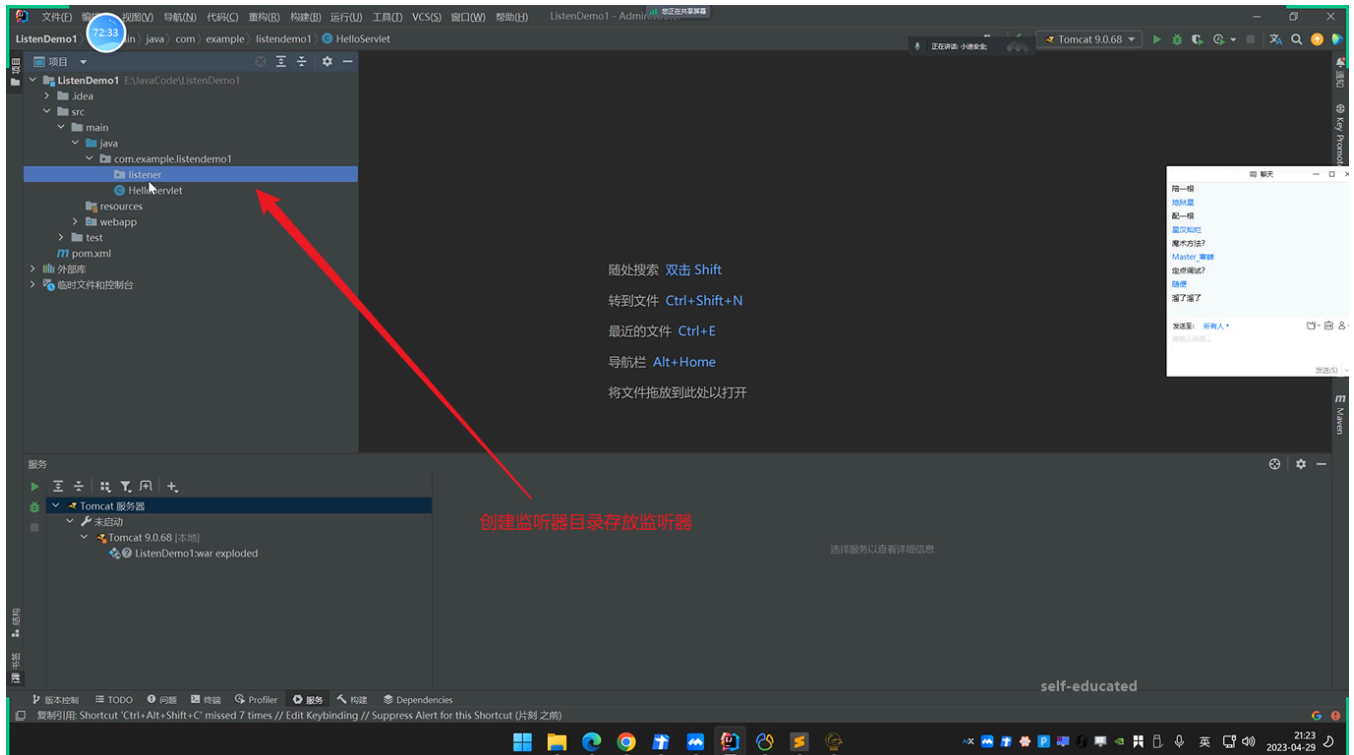


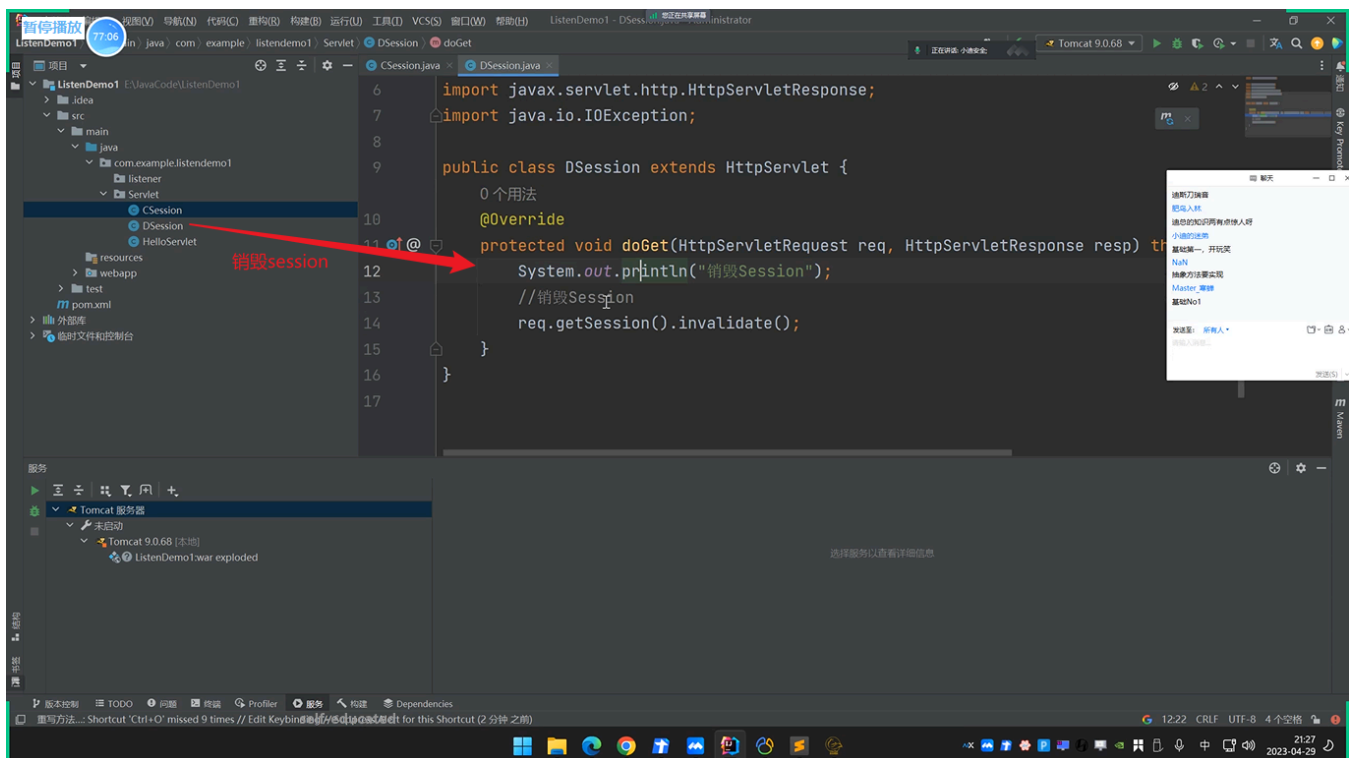
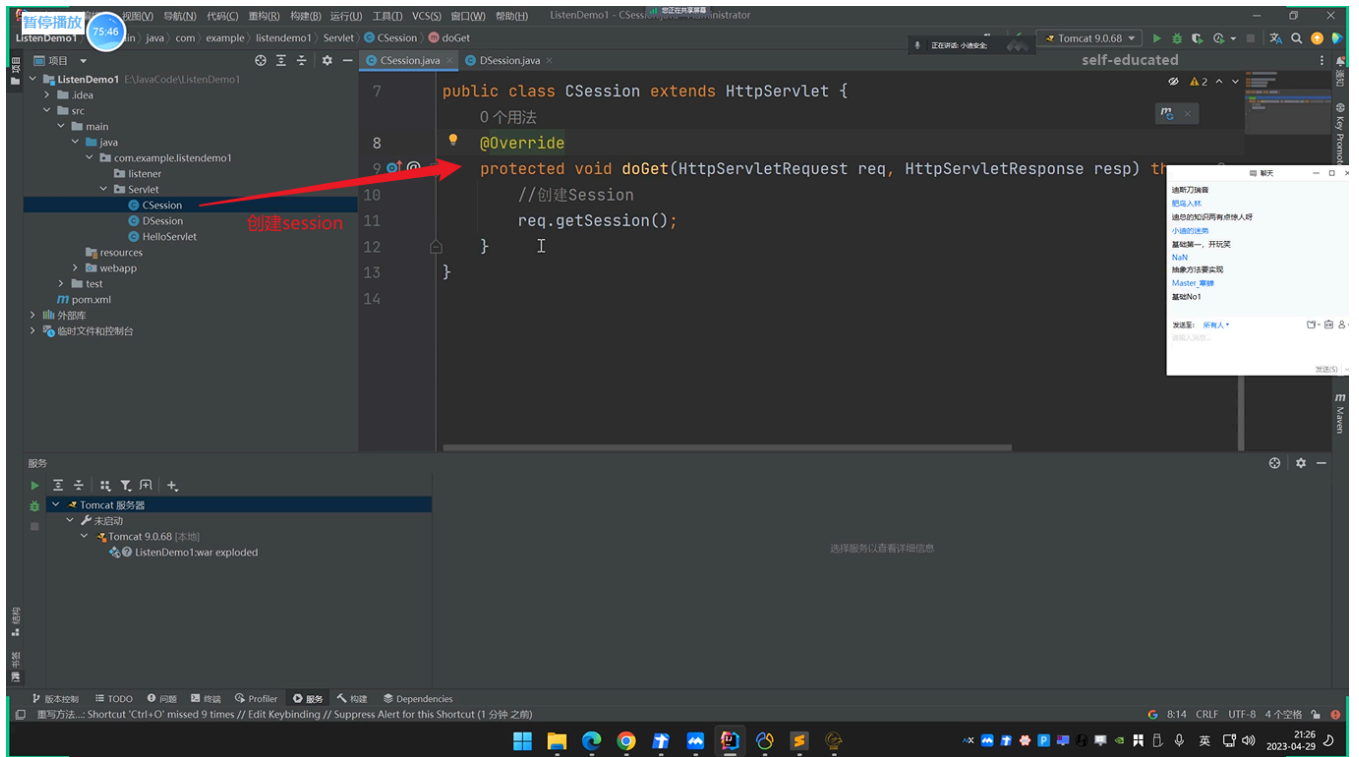


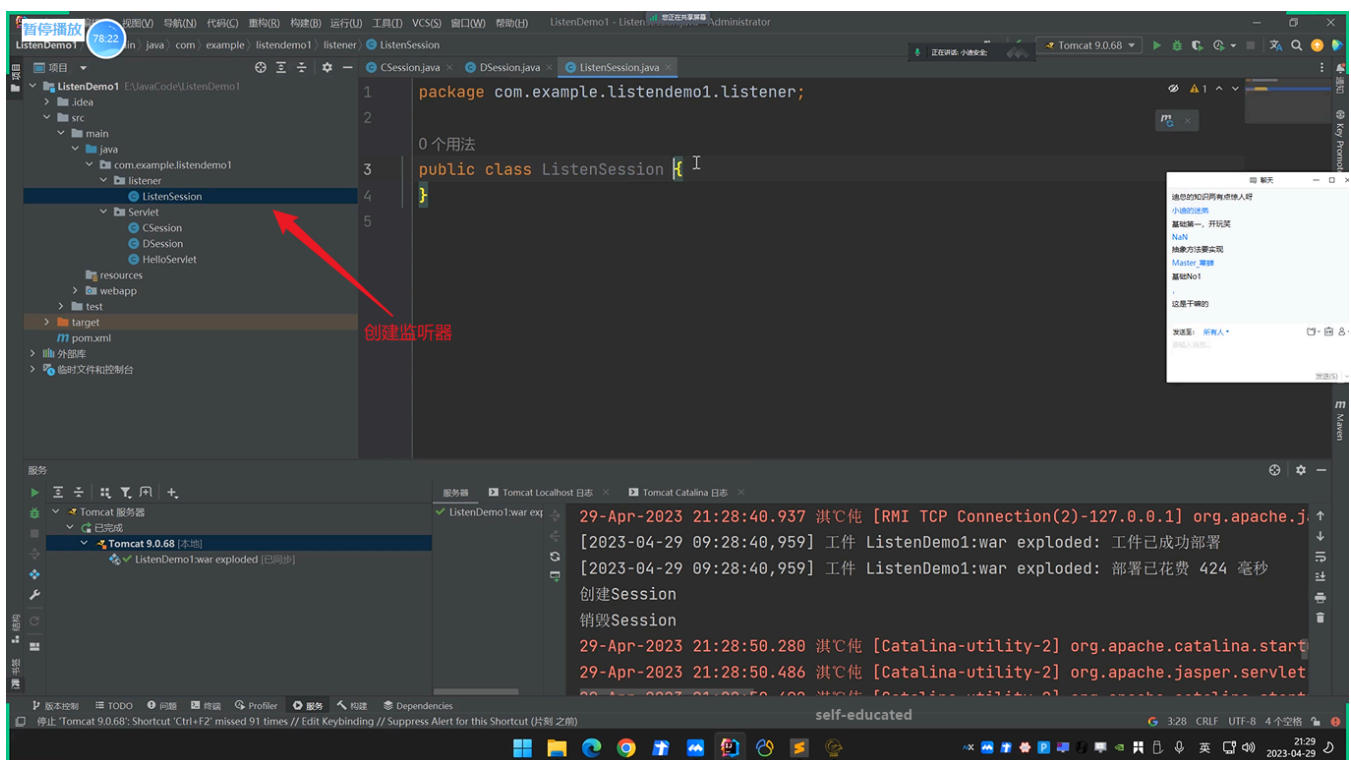
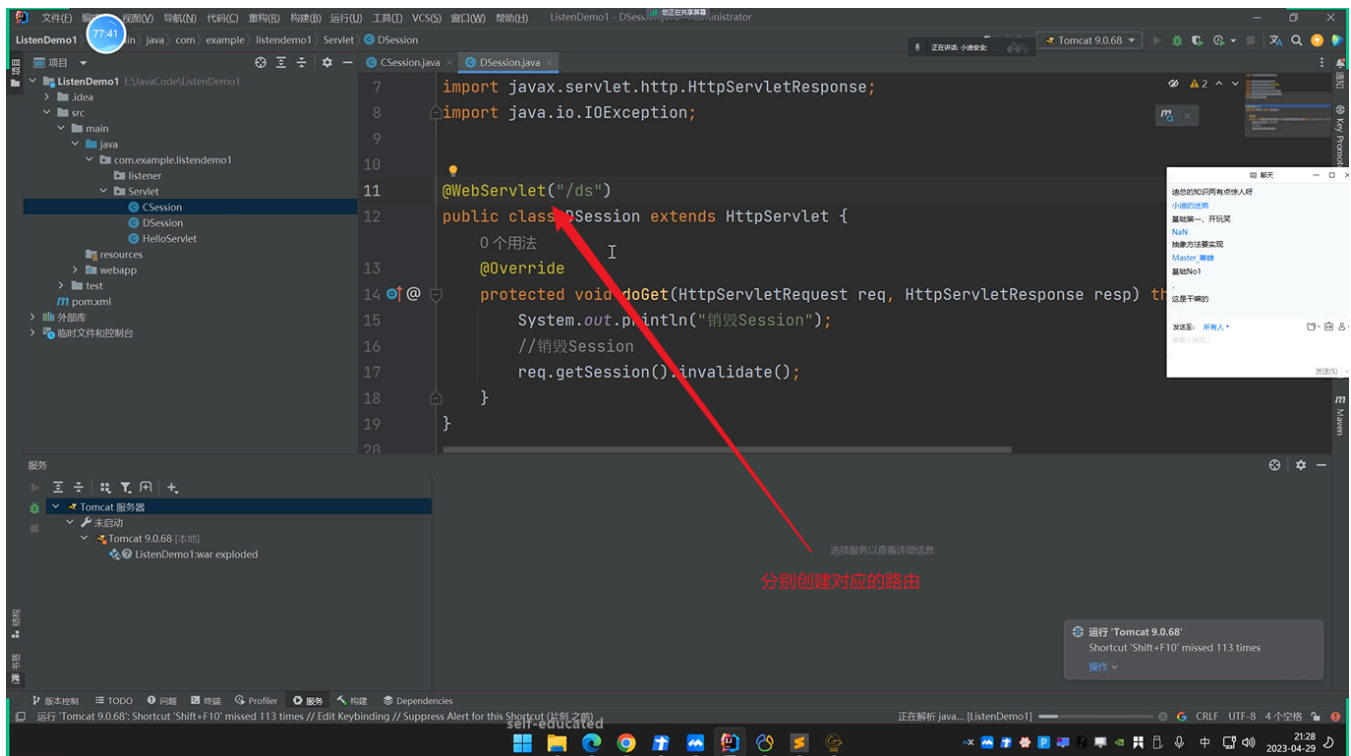
2.3 JavaEE-监听器-Listen

- 1 参考: https://blog.csdn.net/qq_52797170/article/details/124023760
- 2 -监听ServletContext、HttpSession、ServletRequest等域对象创建和销毁事件
- 3 -监听域对象的属性发生修改的事件
- 4 -监听在事件发生前、发生后做一些必要的处理
- 5 1、创建监听器
- 6 2、监听器内置方法
- 7 3、监听器触发流程
- 8 @WebListener
- 9 <listener>
- 10
- 11 </listener>
- 12 4、监听器安全场景
- 13 代码审计中分析执行逻辑触发操作，红队内存马植入，蓝队清理内存马等

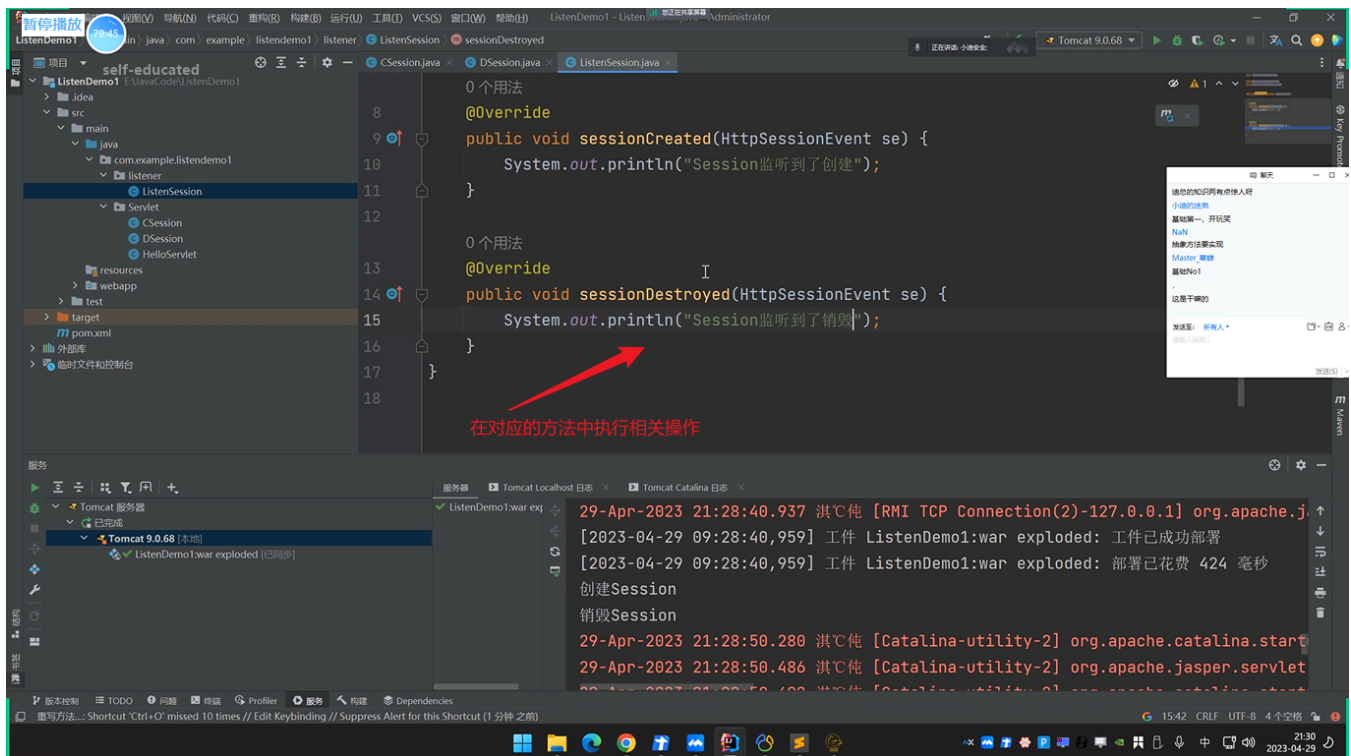
(1) 创建监听器:



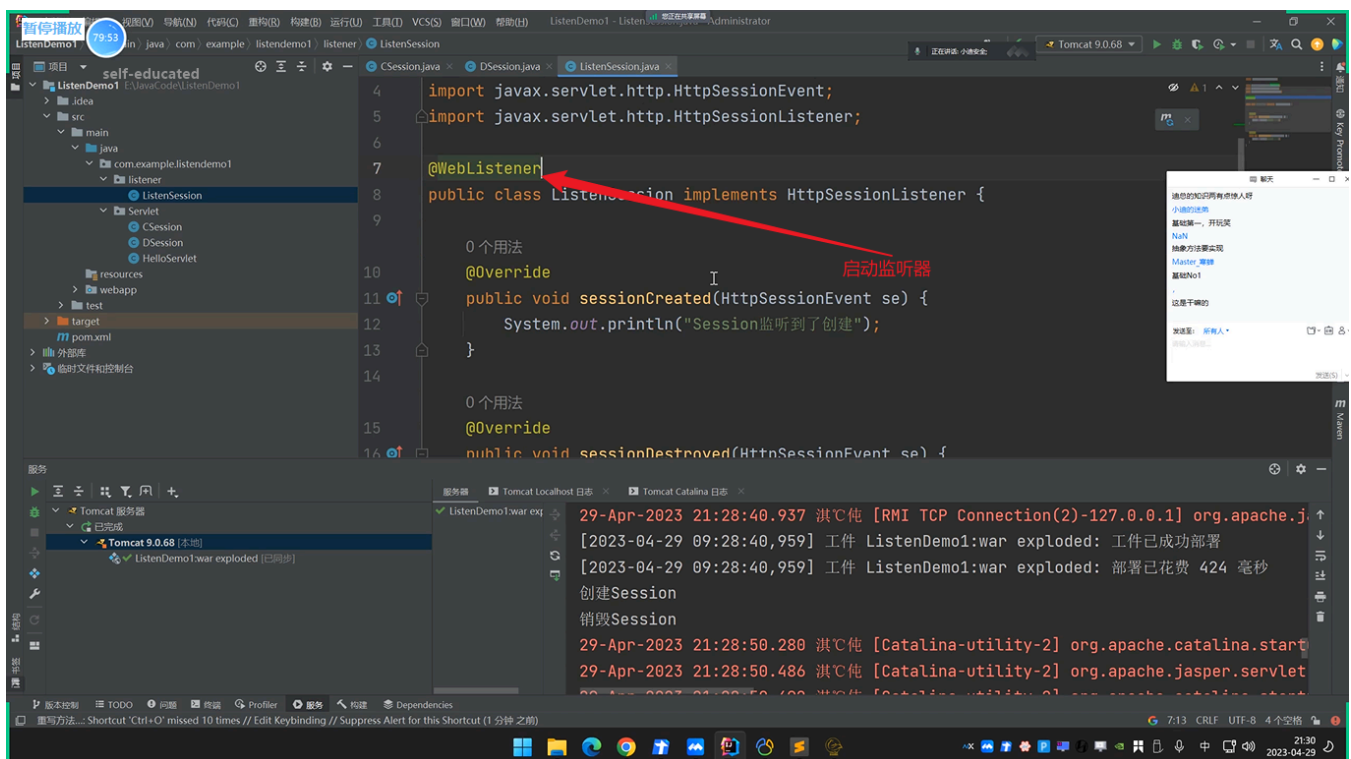


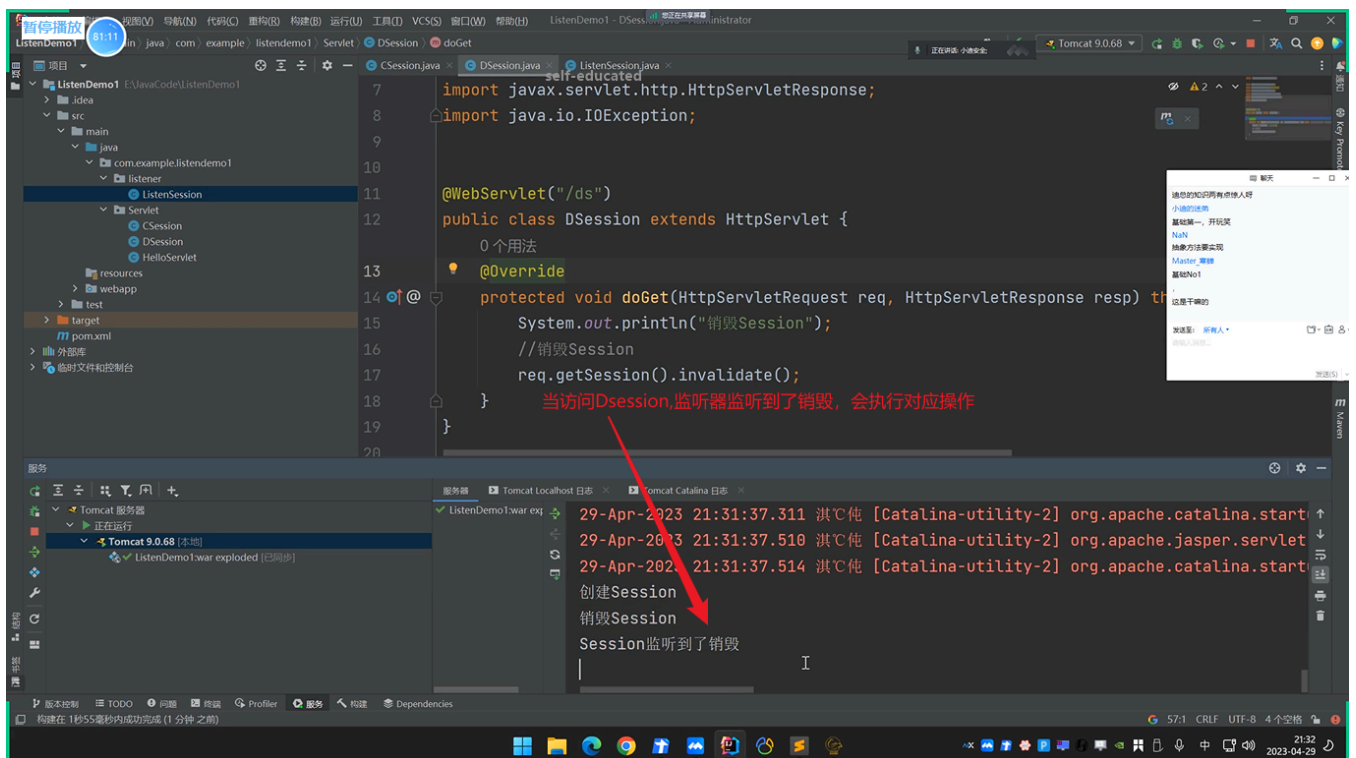
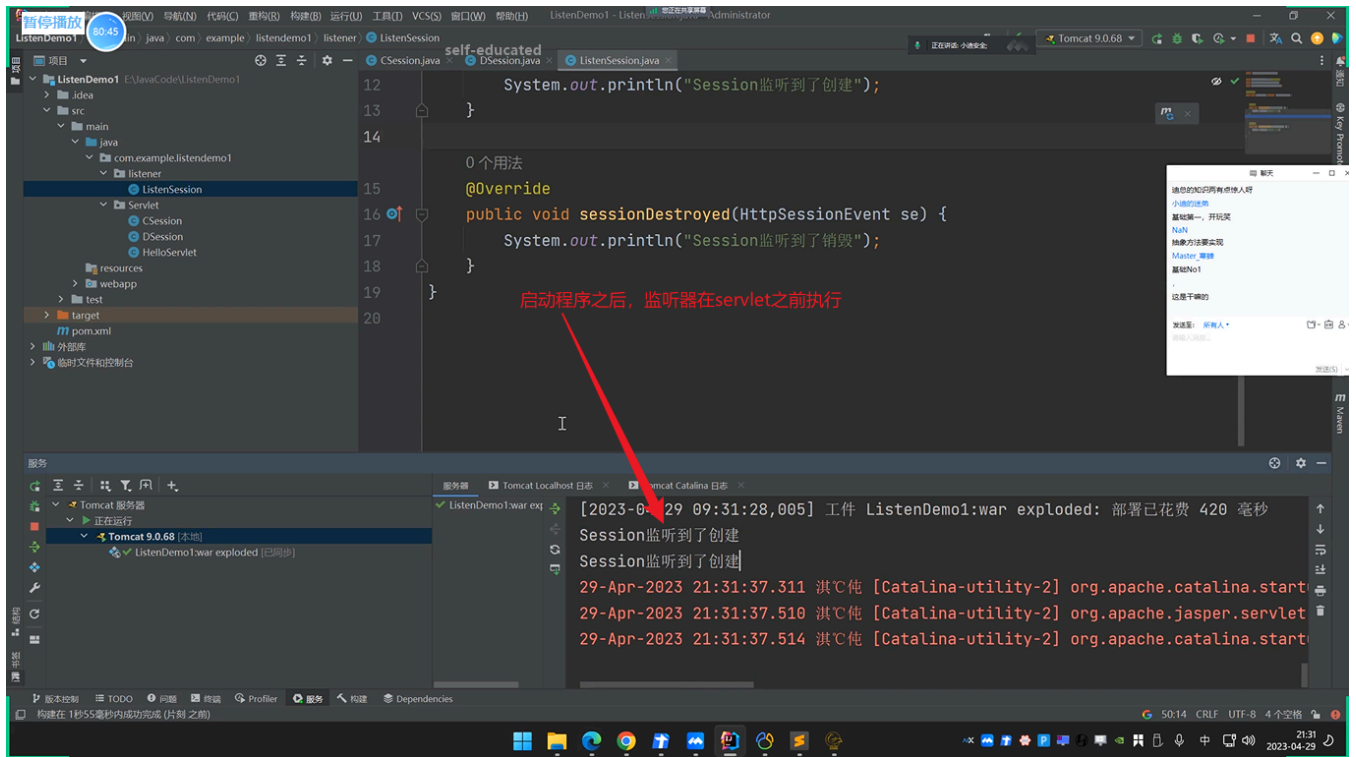


(2) 监听器内置方法:



(3) 监听器触发流程:





新停播放 3301

ListenDemo1 - CSession.java - Administrator

项目 | 代码 | 运行 | 工具 | VCS | 窗口 | 帮助

ListenDemo1 | E:\JavaCode\ListenDemo1

src | main | java | com.example.listendemo1 | Servlet | CSession | doGet

```
7
8 @WebServlet("/cs")
9 public class CSession extends HttpServlet {
10     0 个用法
11     @Override
12     protected void doGet(HttpServletRequest req, HttpServletResponse resp) throws ServletException, IOException {
13         System.out.println("Servlet里面创建Session");
14         //创建Session
15         req.getSession();
16     }
17 }
```

Tomcat 9.0.68

正在构建: 小德安全

Tomcat Localhost 日志 | Tomcat Catalina 日志

ListenDemo1.war exploded [已同步]

Servlet里面销毁Session
监听器监听到了session销毁
Servlet里面创建Session
监听器监听到了session创建

当执行对应的创建销毁, 监听器都会检测并执行相关操作

self-educated

版本控制 | TODO | 问题 | 终端 | Profiler | 服务 | 构建 | Dependencies

构建在 1秒138毫秒内成功完成 (1 分钟之前)

21:35 2023-04-28